

Grafos

Alberto Verdejo
Facultad de Informática (UCM)

Plan de formación OIE

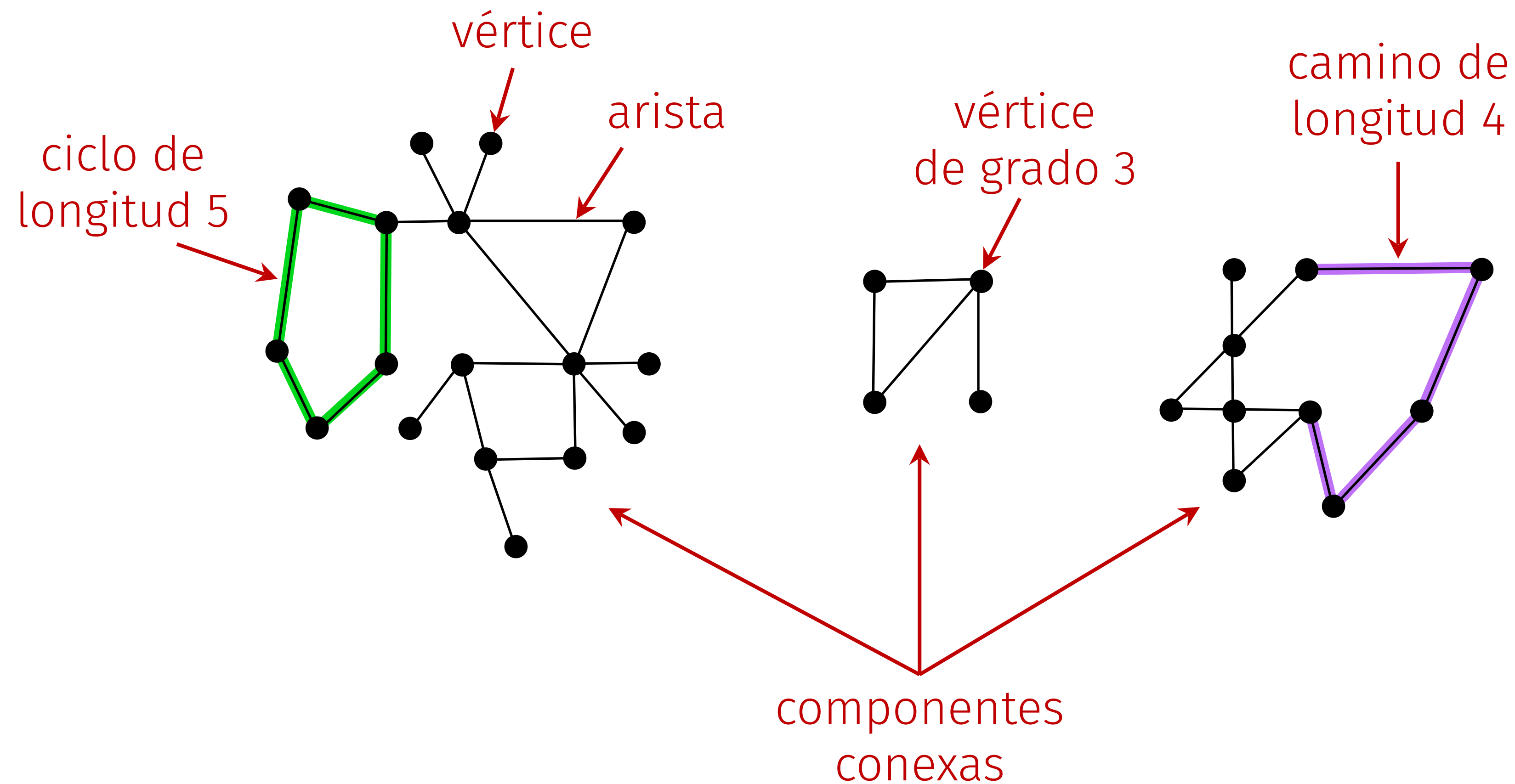
Los grafos sirven para representar elementos y conexiones uno a uno entre ellos.

aplicación	elemento	conexión
mapa	intersección, pueblo	calle, carretera
internet	subred clase C	cable de red
web	página	enlace
red social	persona	amistad
juego	estado del tablero	movimiento legal
circuito	puerta lógica, transistor	cable
red de metro	estación	vía



Terminología

Un grafo es un conjunto de **vértices** y un conjunto de **aristas** que conectan pares de vértices.

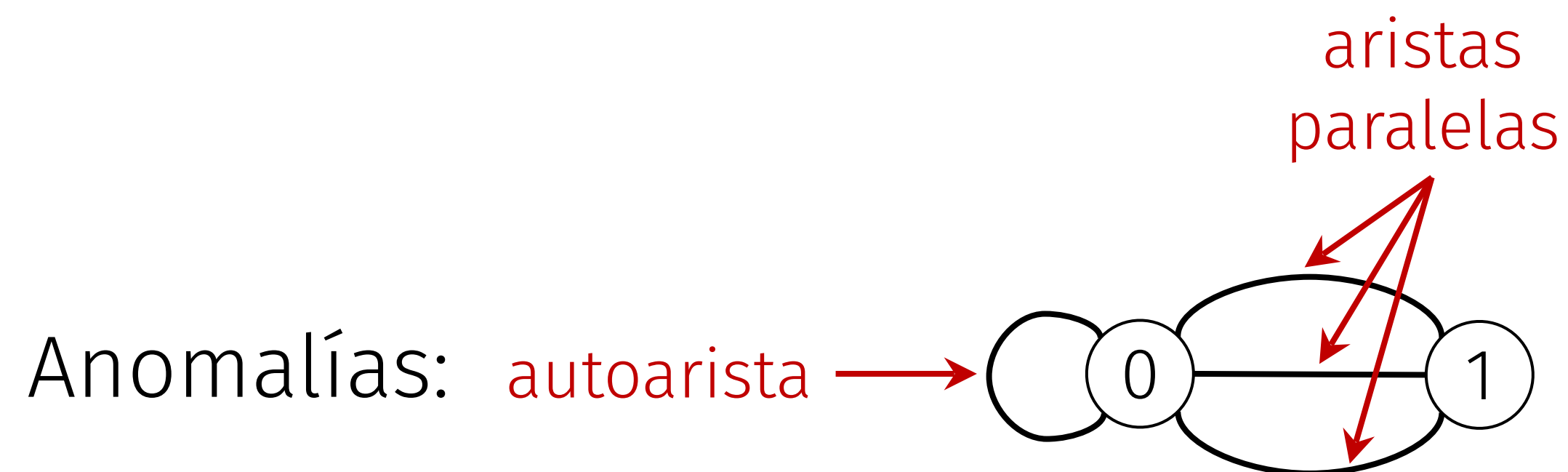
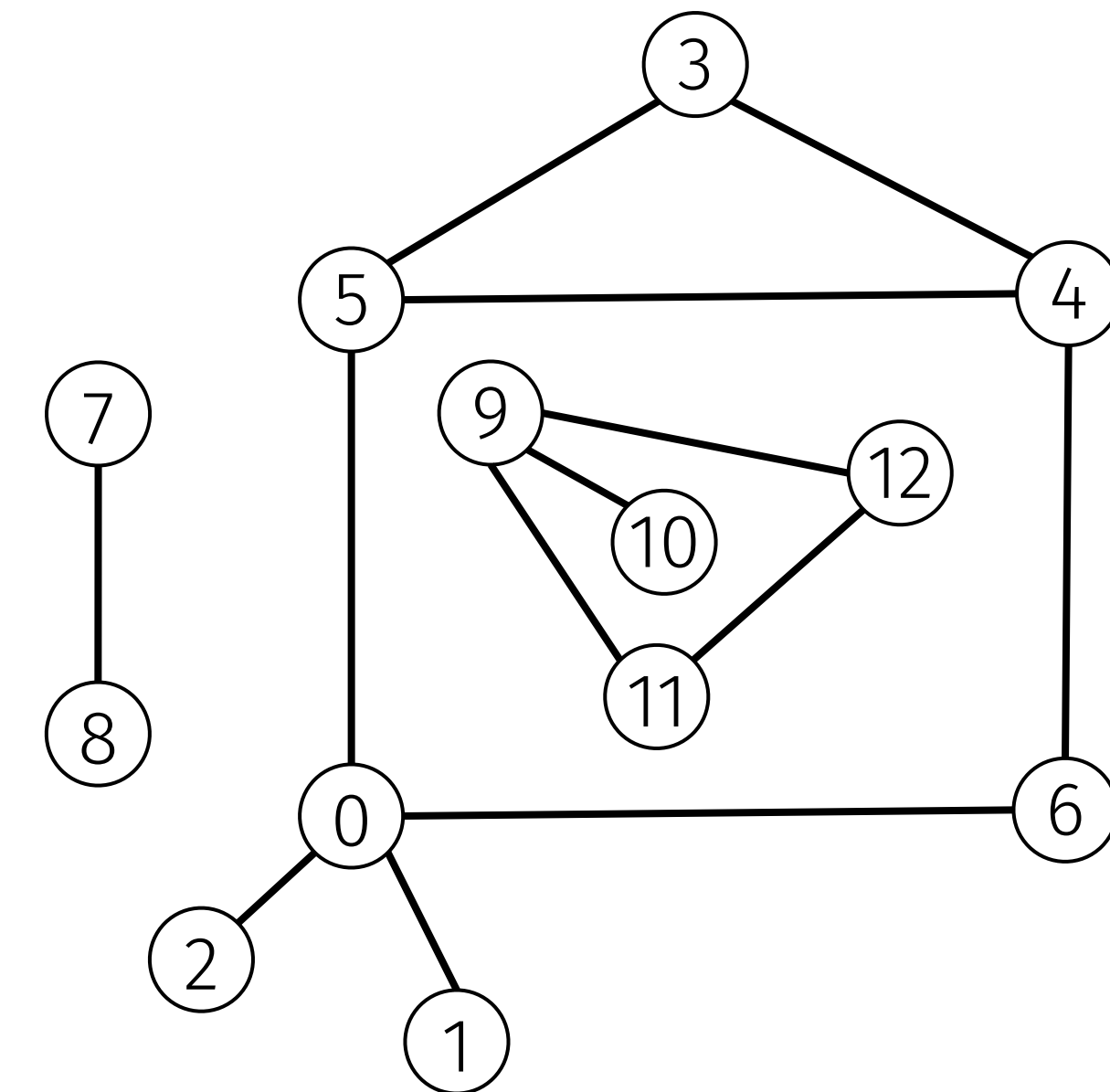
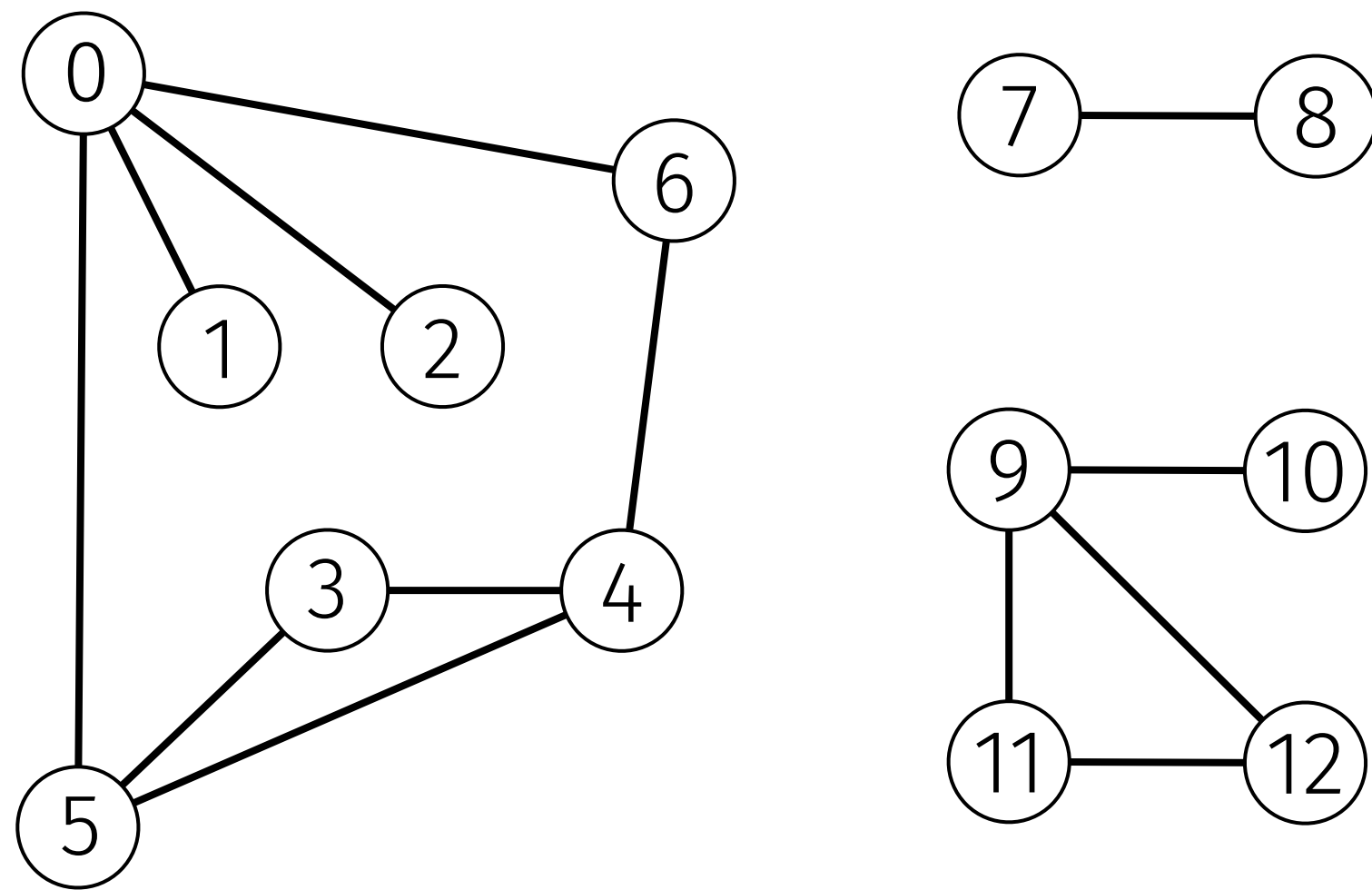




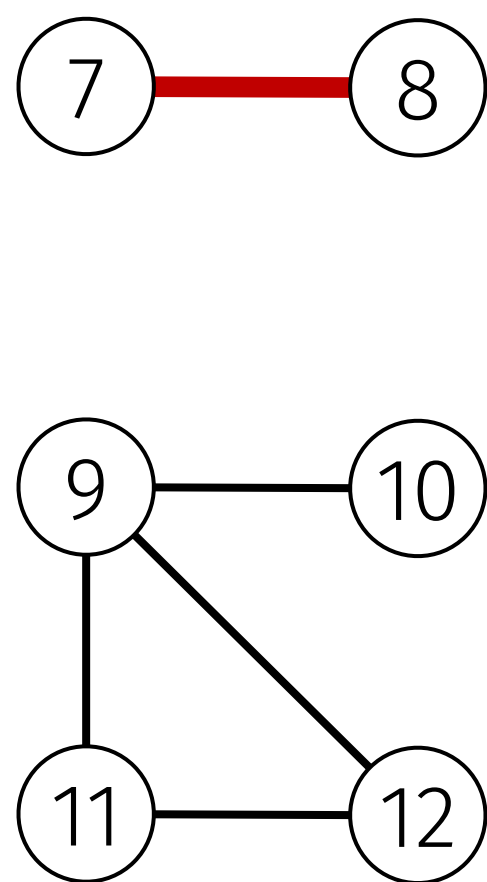
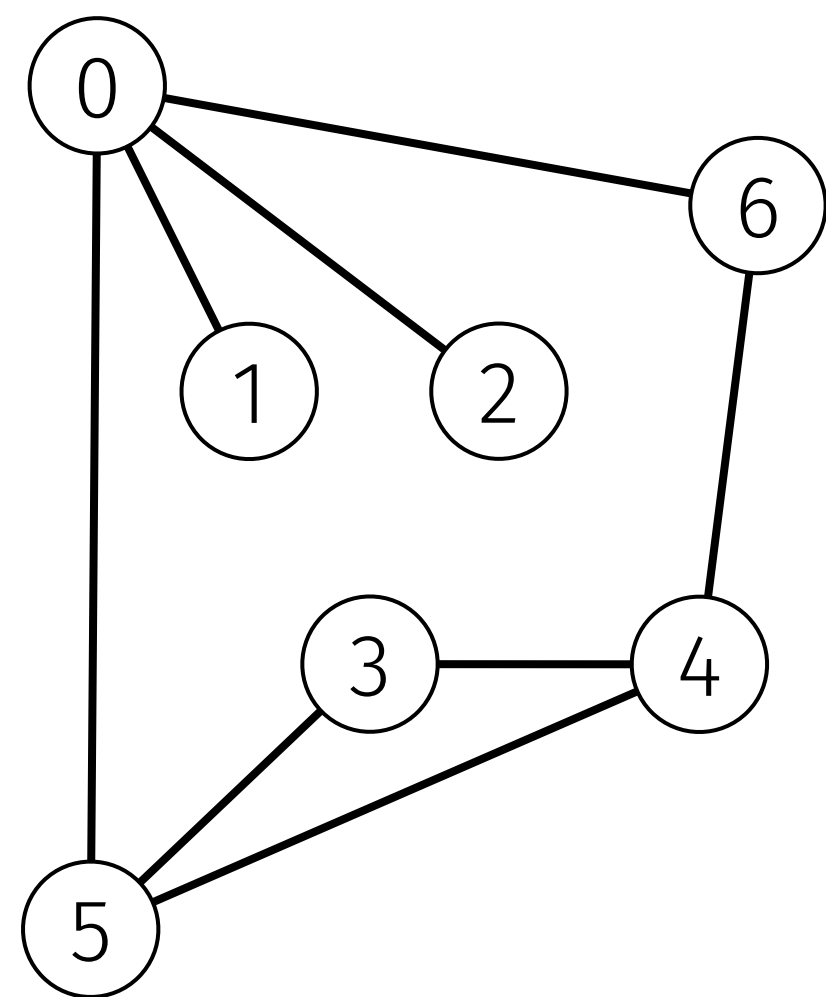
Representación

Representación de un grafo

Un dibujo del grafo nos da intuición sobre su estructura, pero a veces confunde.



Representación: matriz de adyacencia



dos entradas
por arista

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0



Matriz de adyacencia en C++

```
using Grafo = vector<vector<bool>>;

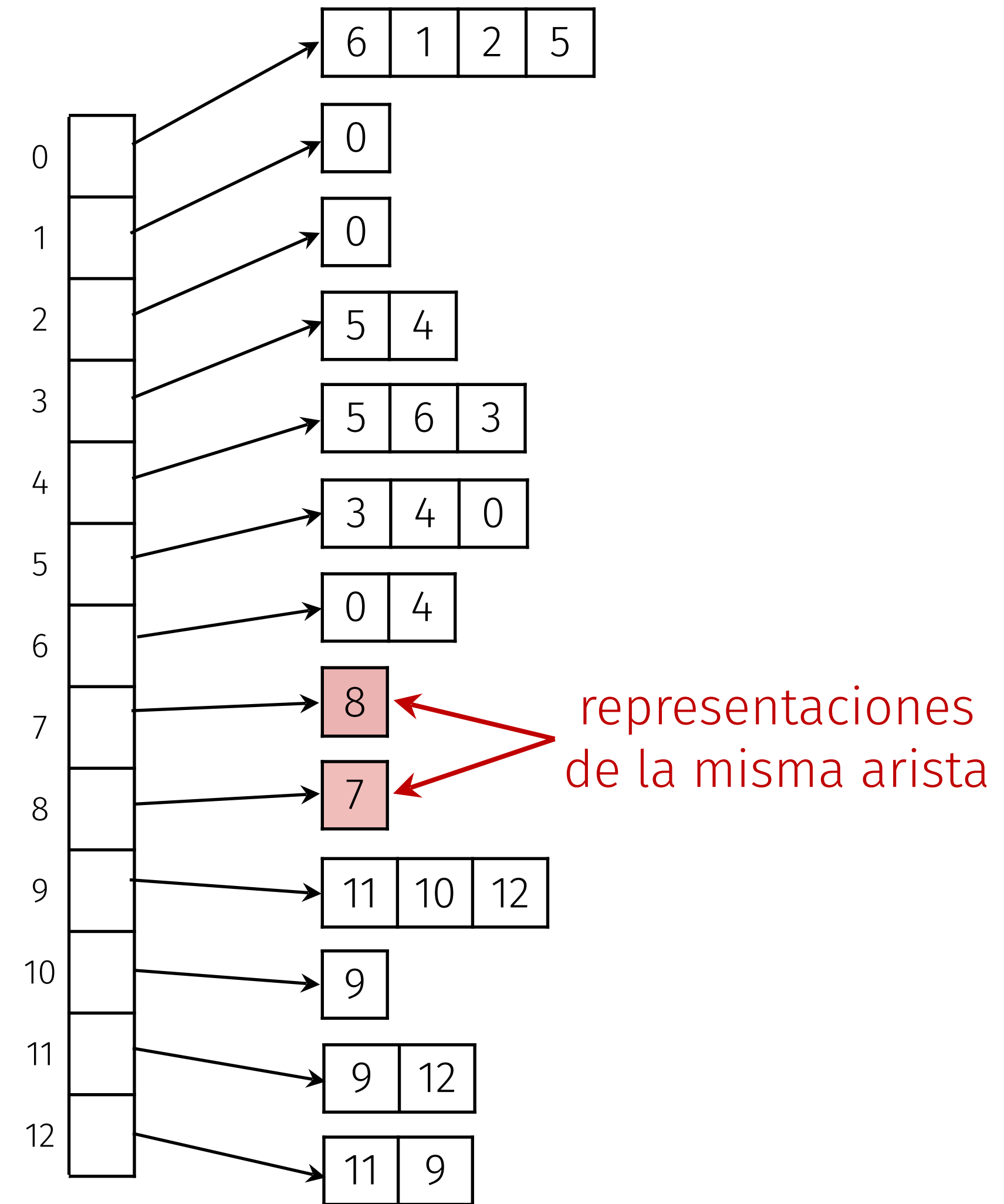
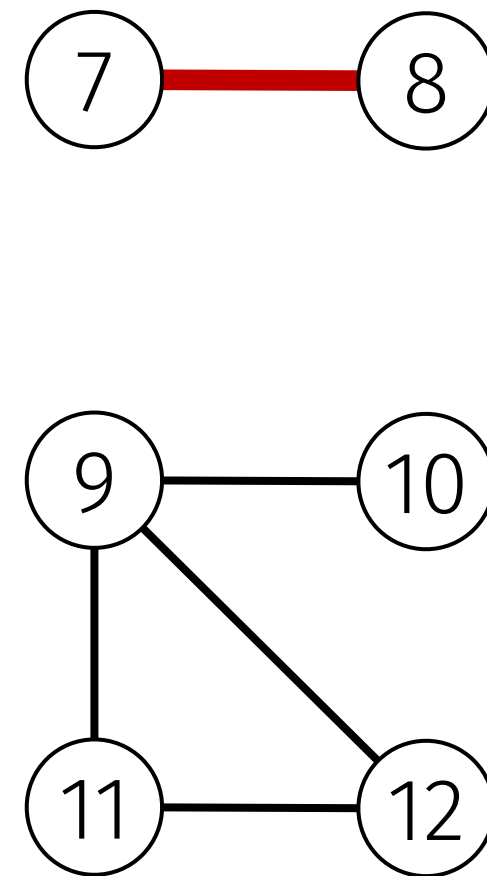
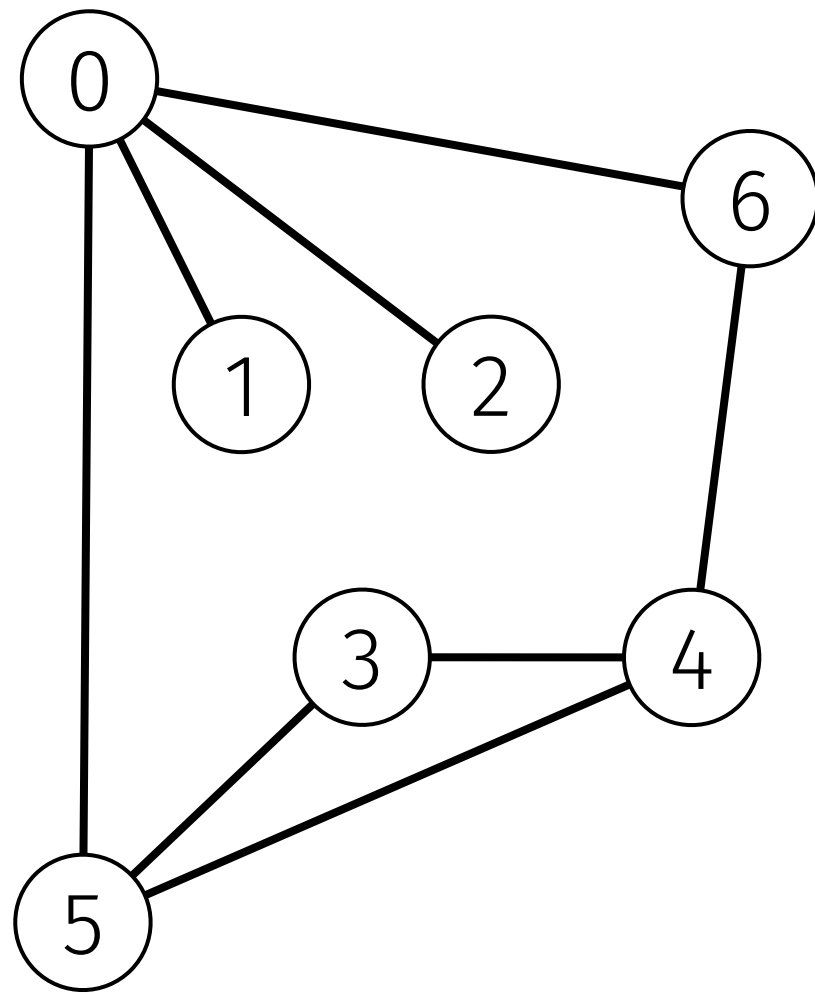
int V = 13; // número de vértices
Grafo g(V, vector<bool>(V, false)); // 0-indexed

// arista 7 - 8

g[7][8] = true;
g[8][7] = true;
```



Representación: listas de adyacentes



Listas de adyacentes en C++

```
using Grafo = vector<vector<int>>;
```

```
int V = 13; // número de vértices
```

```
Grafo g(V); // 0-indexed
```

```
// arista 7 - 8
```

```
g[7].push_back(8);
```

```
g[8].push_back(7);
```



Construcción del grafo

número de vértices

número de aristas

10 9

1 2

3 1

4 8

5 4

3 5

4 6

5 2

7 10

9 10

aristas

```
int V, A;
```

```
cin >> V >> A;
```

```
Grafo g(V); // 0-indexed
```

```
for (int i = 0; i < A; ++i) {
```

```
    int u, v;
```

```
    cin >> u >> v;
```

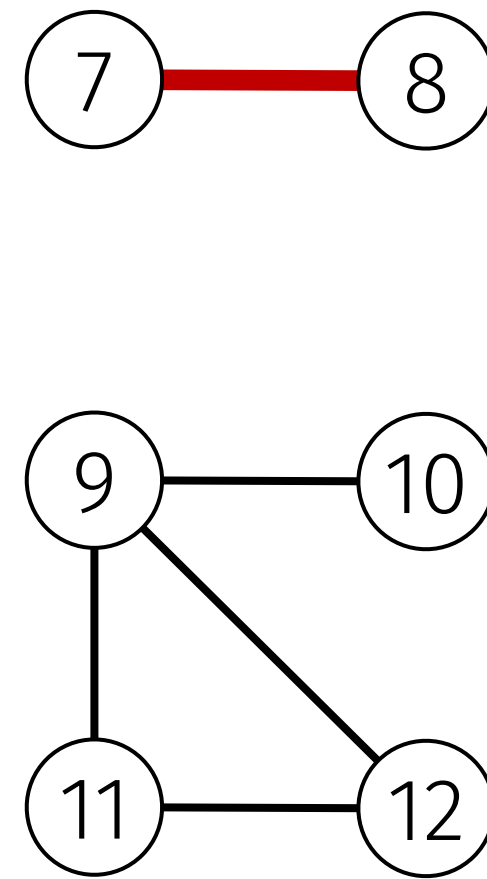
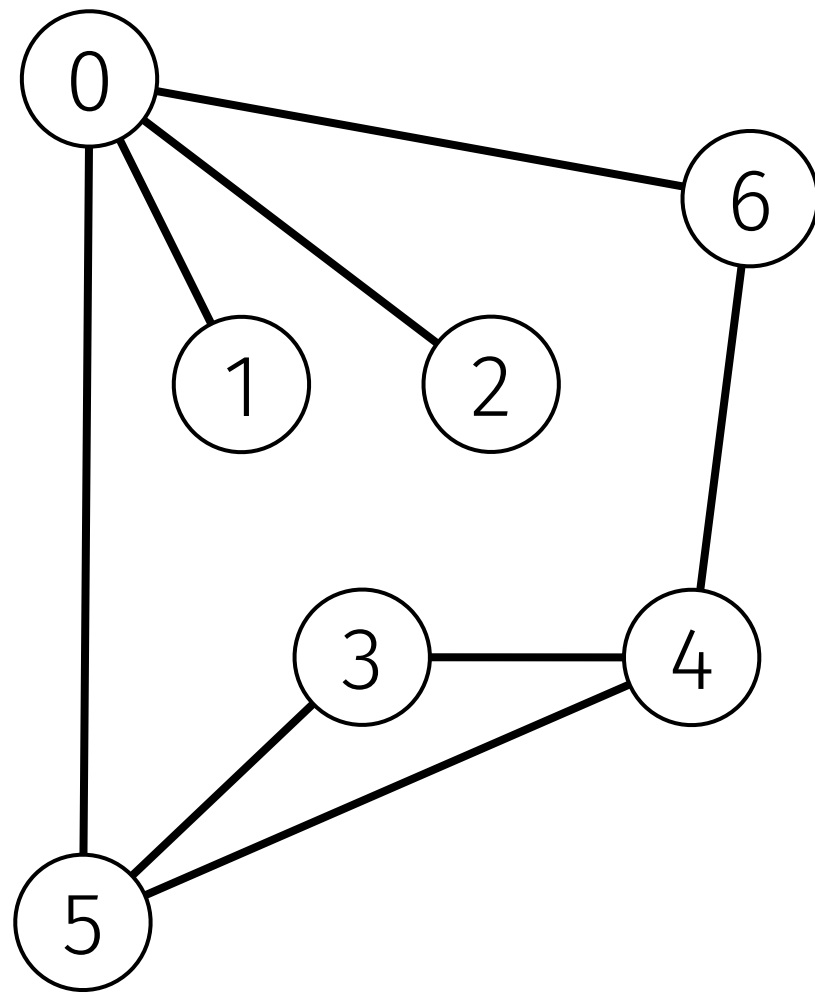
```
    g[u-1].push_back(v-1);
```

```
    g[v-1].push_back(u-1);
```

```
}
```



Representación: lista de aristas



0	1
2	0
0	6
4	6
3	4
5	3
0	5
5	4
7	8
9	10
9	12
11	12
11	9



Lista de aristas en C++

```
using Grafo = vector<pair<int,int>>;
```

```
int A; // número de aristas  
Grafo g;
```

```
// arista 7 - 8  
g.push_back({7, 8});
```

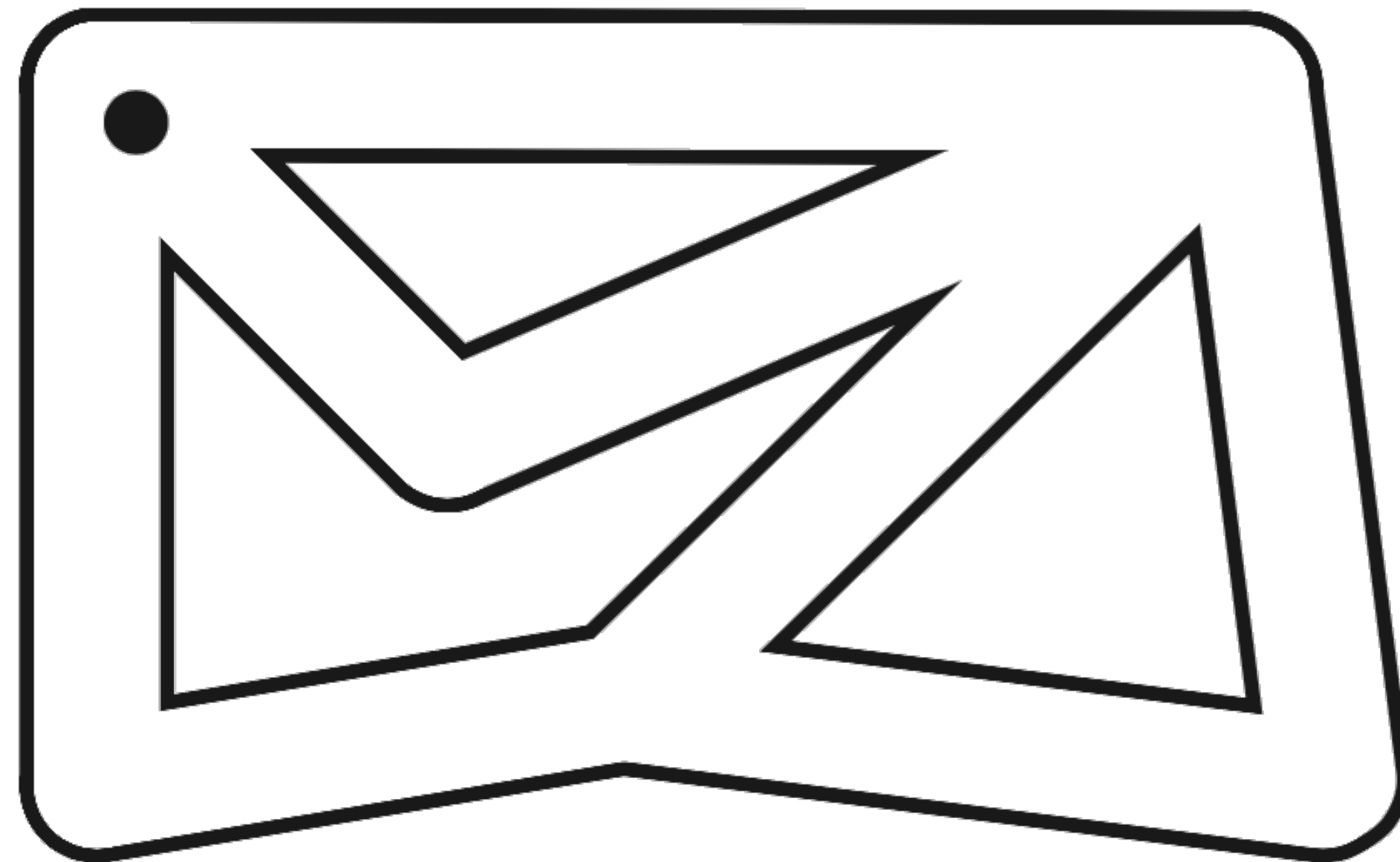




*Recorridos de grafos:
recorrido en profundidad*

Recorrido en profundidad

El recorrido o búsqueda en profundidad (en inglés, *depth-first search*, *DFS*) imita la resolución de un laberinto.

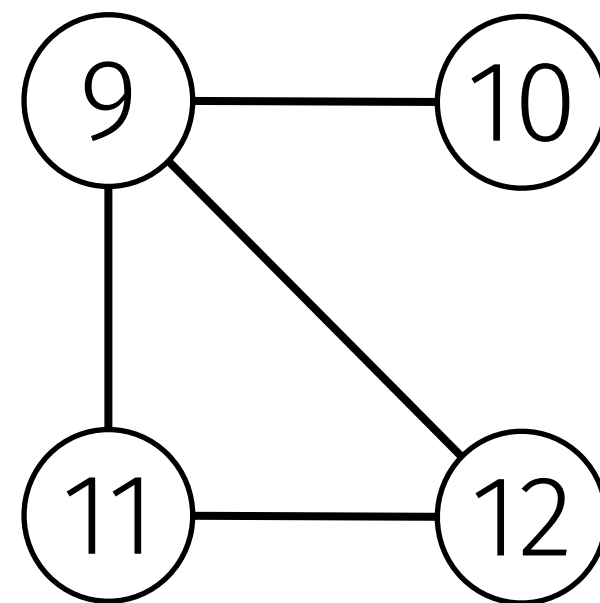
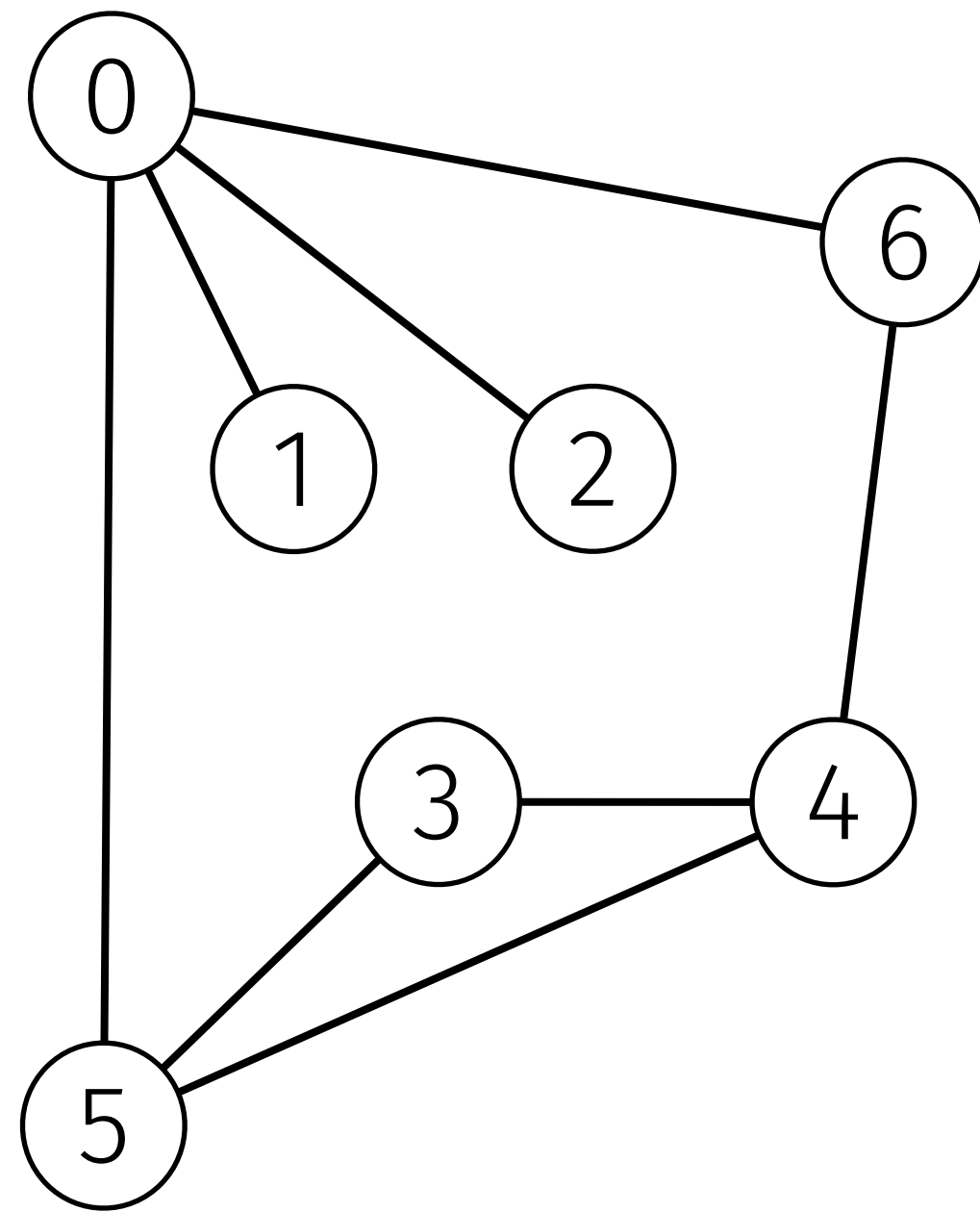


Recorrido en profundidad

- ▶ Para recorrer un grafo utilizamos un algoritmo recursivo que va visitando vértices.
- ▶ Visitar un vértice v consiste en:
 - ▶ *hacer algo con él;*
 - ▶ marcarlo como visitado; y
 - ▶ visitar (recursivamente) todos los vértices adyacentes a v aún no visitados.



Recorrido en profundidad



Recorrido en profundidad en C++

```
using Grafo = vector<vector<int>>;
```

```
Grafo g;
```

```
vector<bool> visit;
```

```
void dfs(int v) {  
    visit[v] = true;  
    for (int w : g[v])  
        if (!visit[w])  
            dfs(w);  
}
```



Problema: Los amigos de mis amigos son mis amigos

En esta ciudad vive una serie de personas, y sabemos que algunas de ellas son amigas entre sí. De acuerdo con el refrán que dice “*Los amigos de mis amigos son mis amigos*”, sabemos que si A y B son amigos y B y C son amigos, entonces también son amigos A y C .

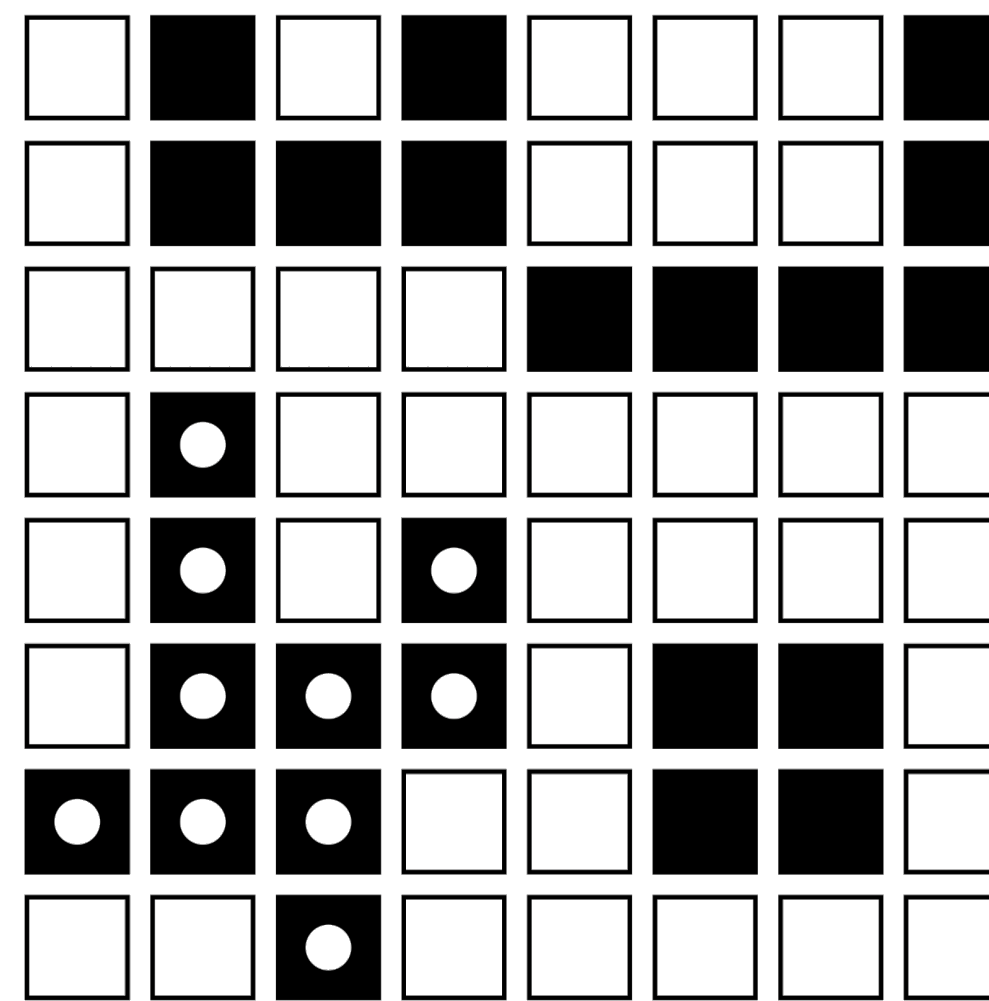


Nuestra misión consiste en contar las personas en el grupo de amigos más grande.



Problema: Detección de manchas negras

Dado un *bitmap* de píxeles blancos y negros, queremos saber el número de manchas negras que contiene y el tamaño (número de píxeles) de la mayor.



En esta imagen aparecen 4 manchas y la mancha más grande (marcada con puntos blancos) tiene 10 píxeles.



Problema: Detección de manchas negras

número F de filas



8

número C de columnas

8

-#-#----

-####----

-----#####

-#-----

-#-#-----

-####-##-

###- -##-

- -#-----

} mapa de
"pixeles"





*Recorridos de grafos:
recorrido en anchura*

Recorrido en anchura

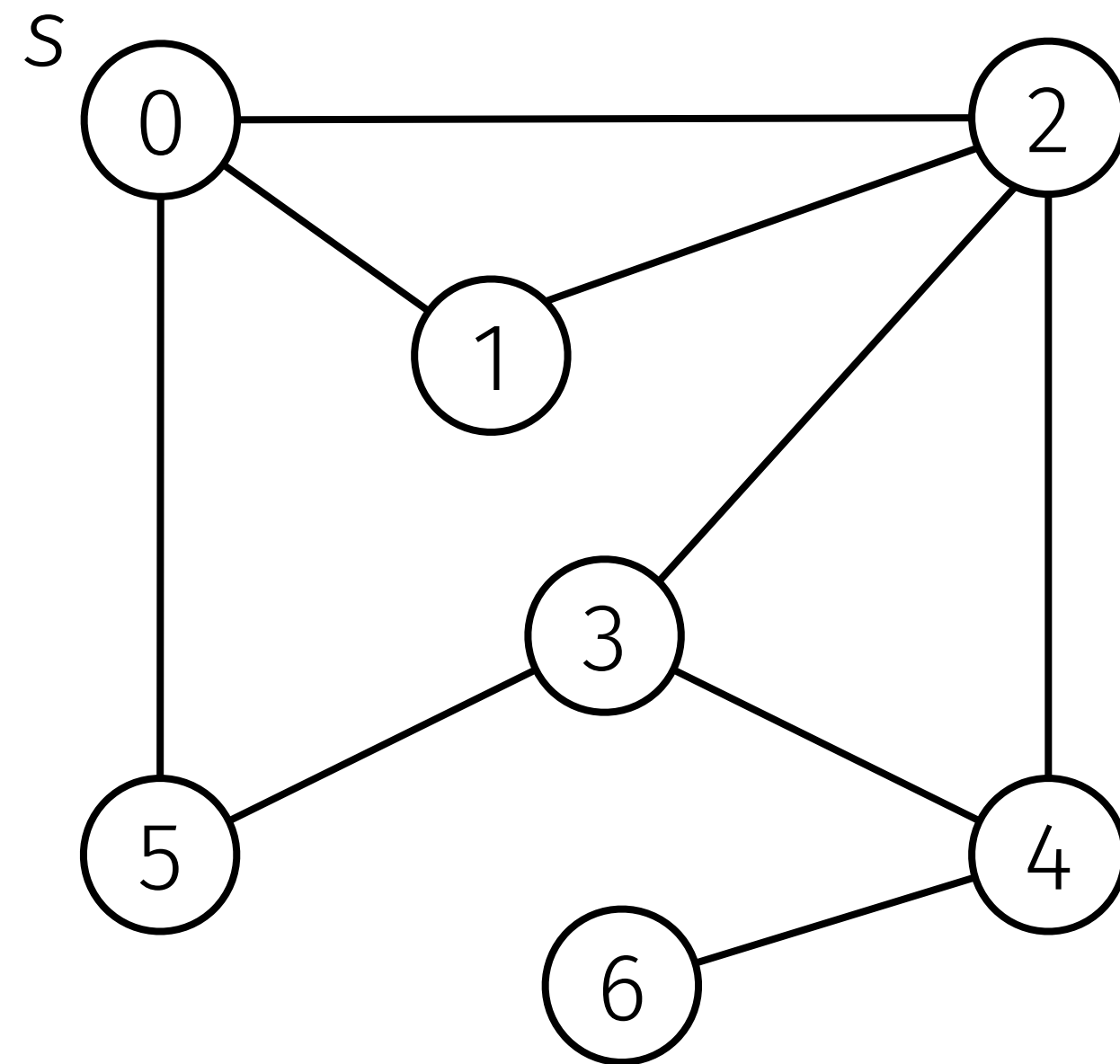
En ocasiones estamos interesados en encontrar el **camino más corto** desde un origen s a otro vértice v (o a todos los vértices conectados a s).

El **recorrido en anchura** (en inglés, *breadth-first search*, *BFS*) logra eso: primero visita todos los vértices alcanzables siguiendo una arista (a distancia 1); luego visita todos los vértices alcanzables utilizando dos aristas (a distancia 2); y así sucesivamente.

Para lograrlo utiliza una **cola** donde guardar los vértices alcanzados pero que aún no se han explorado sus adyacentes.



Recorrido en anchura



Recorrido en anchura en C++

```
int V, A;
Grafo g;

vector<int> bfs(int s) {
    vector<int> dist(V, -1); dist[s] = 0;
    queue<int> q; q.push(s);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int w : g[v])
            if (dist[w] == -1) {
                dist[w] = dist[v] + 1;
                q.push(w);
            }
    }
    return dist;
}
```



Problema: Haciendo trampas en Serpientes y escaleras

